



南開大學

Nankai University

计算机学院
并行程序设计期末报告

MPI 多进程实验

姓名：刘宇泽

学号：2411334

专业：计算机科学与技术

2026 年 6 月 3 日

目录

1 问题描述	1
2 数据集	1
3 算法设计	1
3.1 MPI IVF	1
3.1.1 算法原理	1
3.1.2 流程图	2
3.1.3 伪代码	2
3.1.4 复杂度与理论耗时分析	3
3.2 分片 HNSW	4
3.2.1 算法原理	4
3.2.2 关键数据结构	5
3.2.3 流程图	5
3.2.4 伪代码	6
3.2.5 复杂度与理论耗时分析	7
3.2.6 工程要点	8
3.3 IVF+HNSW	8
3.3.1 算法原理	8
3.3.2 关键数据结构	9
3.3.3 流程图	9
3.3.4 伪代码	9
3.3.5 复杂度与理论耗时分析	11
3.3.6 工程要点	12
4 通信模式设计	13
4.1 阻塞通信	13
4.2 非阻塞通信	13
4.3 单边通信	13
5 实验参数配置	13
5.1 MPI IVF	13
5.2 分片 HNSW	14
5.3 IVF+HNSW	14
5.4 测试矩阵	15
6 实验结果与分析	15
6.1 MPI IVF 参数与系统行为分析	15
6.1.1 nprobe 与 recall 的关系 (thread_mode=openmp, threads=4)	15
6.1.2 线程模式与延迟 (nlist=32, nprobe=8)	15
6.2 通信方式与混合并行分析	16
6.3 分片 HNSW 性能分析	17

6.4	IVF+HNSW 性能分析	18
6.5	综合对比	19
6.5.1	Recall-Latency 帕累托前沿	19
6.5.2	线程模式加速比对比	20
6.5.3	算法延迟量级对比	20
7	实验总结	20
A	全部实验原始数据	22

1 问题描述

近似最近邻搜索 (ANNS) 在大规模向量检索中面临明显的计算与访存瓶颈。对于 DEEP100K 数据集, 基向量规模为 100,000, 维度为 96, 若直接采用串行全量扫描, 则每个查询都需要遍历全部向量并计算大量内积, 随着查询数增加, 整体延迟难以满足实际检索需求。

本次实验进一步探索 **MPI 多进程并行化** 在 ANN 查询阶段中的可行性。和前面 pthread / OpenMP 实验一样, 本次依旧重点关注查询阶段, 而非索引构建阶段。核心问题可归结为:

1. 如何将 base 数据合理划分到多个 MPI 进程;
2. 如何设计广播、局部搜索和最终 merge/reduce 的流程;
3. 如何将 MPI 与 OpenMP / pthread 进一步结合, 形成混合并行;
4. 如何覆盖阻塞、非阻塞、单边通信, 以及图索引方案, 并如实分析其效果。

2 数据集

数据集	规模	维度
DEEP100K.base	100,000	96
DEEP100K.query	100,000	96
DEEP100K.gt	100,000 × 100	-

表 1: 数据集规模

3 算法设计

3.1 MPI IVF

3.1.1 算法原理

MPI IVF 的核心思想是: 将 base data 按 rank 做连续分片, 每个进程仅维护自己分片上的 IVF 索引; 查询到来后由 rank0 广播给其他进程, 各进程分别完成局部 coarse search 与 fine search, 得到局部 top-k, 最后再由 rank0 负责 merge 成全局 top-k 并计算 recall。

与单机 IVF 相比, MPI IVF 的本质变化不在于局部索引逻辑, 而在于把”候选集来源”从单机全量 base, 变成了多个 rank 的局部分片。因此它天然包含两级开销:

1. **通信开销**: query 广播与局部结果汇总;
2. **计算开销**: 每个 rank 上的局部 IVF 搜索。

在本次实现中, 局部 IVF 搜索支持三种 rank 内线程模式:

- **none**: 单线程;
- **openmp**: 复用已有 OpenMP IVF 实现;
- **pthread**: 复用已有 pthread IVF 实现。

因此, MPI IVF 可以自然扩展为两级并行结构:

- 进程间并行: MPI 分片;
- 进程内并行: OpenMP / pthread。

3.1.2 流程图

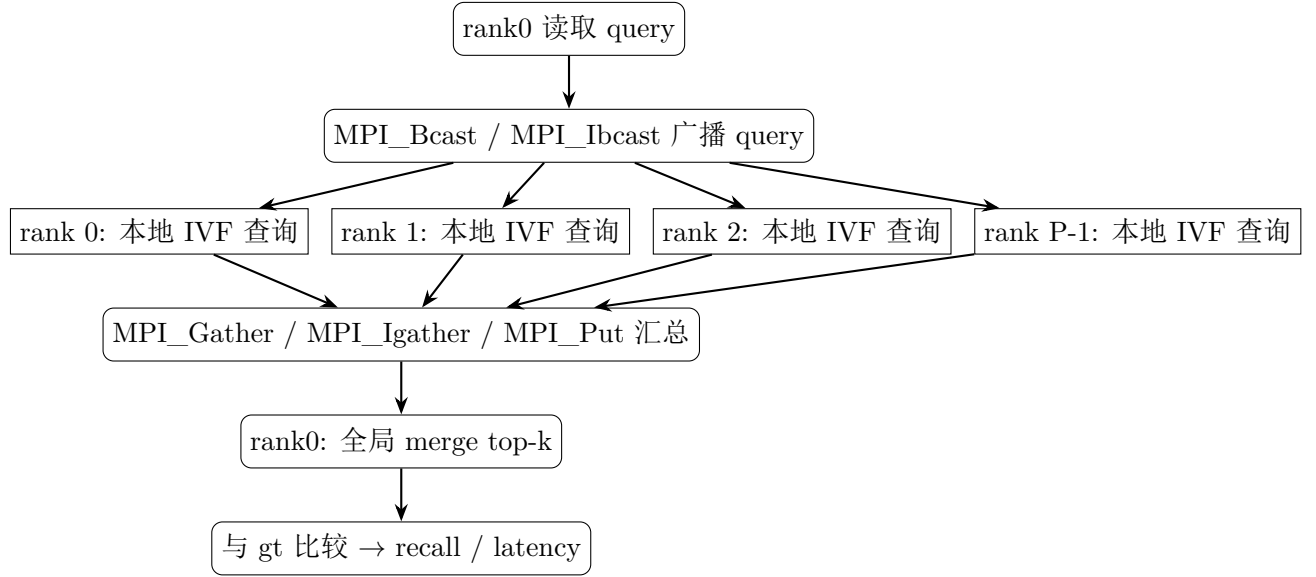


图 3.1: MPI IVF 主方案流程图

3.1.3 伪代码

Algorithm 1 MPIIVFSearch

Input: $\text{base}[N][d]$, $\text{query}[Q][d]$, nlist , nprobe , k , P (processes), T (threads per process)

Output: recall / latency statistics for the Q queries

```

1: // Build phase
2: for  $p = 0$  to  $P - 1$  do
3:    $\text{shard}[p] = \text{base}[p \cdot N/P : (p + 1) \cdot N/P]$ 
4:    $\text{kmeans}(\text{shard}[p], \text{nlist}, \text{centroids}[p])$ 
5:   for  $c = 0$  to  $\text{nlist}-1$  do
6:      $\text{inv\_list}[p][c] = \{x \in \text{shard}[p] \mid \text{nearest}(\text{centroids}[p][c], x)\}$ 
7:   end for
8: end for
9: // Search phase
10: for  $i = 0$  to  $Q - 1$  do
11:    $q = \text{query}[i]$ 
12:    $\text{Bcast}(q)$  // query broadcast
13:   for  $p = 0$  to  $P - 1$  in parallel do
14:     // 阶段一: 局部粗排
15:     for  $c = 0$  to  $\text{nlist}-1$  do
16:        $\text{coarse\_d}[c] = \text{inner\_product\_neon}(q, \text{centroids}[p][c], d)$ 

```

```

17:     end for
18:     selected = top-nprobe(coarse_d)
19:     // 阶段二：局部精排
20:     candidates =  $\bigcup_{c \in \text{selected}} \text{inv\_list}[p][c]$ 
21:     #pragma omp parallel for schedule(static)
22:     for j in candidates do
23:         d = inner_product_neon(q, base[j], d)
24:         maintain local_topk[p][tid] as min-heap of size k
25:     end for
26: end for
27: Gather(local_topk)
28: global_topk[i] = merge_k_way(local_topk[0..P-1], k)
29: update recall / latency
30: end for
31: compute average recall, average latency, max latency

```

3.1.4 复杂度与理论耗时分析

记 base 总规模为 N ，向量维度为 d ，进程数为 P ，每个进程内线程数为 T 。对于一次查询：

1. 通信侧

- 广播 query 一次： $\alpha + \beta \cdot d$ (α 为固定延迟， β 为单位字节传输时间)
- 汇总局部 top-k： $\alpha + \beta \cdot P \cdot k \cdot (\text{id} + \text{score})$

2. 计算侧

- 每个 rank 上 coarse search： $O(\text{nlist} \cdot d)$ ，随 nlist 线性增长，与 N 无关
- 每个 rank 上 fine search：
 - 每 list 候选约 $N/(P \cdot \text{nlist})$
 - 探测 nprobe 个 list 的候选规模约 $\text{nprobe} \cdot N/(P \cdot \text{nlist})$
 - 单次 SIMD L2 距离耗时 $T_d \approx \theta_d \cdot d$
 - 单 rank 上 fine search 耗时近似： $T_{\text{fine}} \approx \text{nprobe} \cdot N/(P \cdot \text{nlist}) \cdot T_d$
 - rank 内 T 个线程加速后近似 T_{fine}/T

3. 合并：merge_topk($P \cdot k$) 复杂度 $O(P \cdot k \cdot \log k)$

4. 总耗时模型

$$\begin{aligned}
 T_{\text{query}} \approx & \underbrace{2\alpha + \beta \cdot d}_{\text{通信}} + \underbrace{\text{nlist} \cdot T_d}_{\text{粗排}} \\
 & + \underbrace{\text{nprobe} \cdot N/(P \cdot \text{nlist}) \cdot T_d/T}_{\text{精排}} + \underbrace{P \cdot k \cdot T_d}_{\text{合并}}
 \end{aligned} \tag{1}$$

5. 理论结论

- P 增大 $\rightarrow$ 计算侧降低: 计算侧系数 $1/P$, 通信几乎与 P 无关; P 很大时通信与 merge 成为新瓶颈
- $nlist$ 增大 $\rightarrow$ coarse 增长、fine 减小: $nlist$ 与 $nprobe$ 存在 trade-off
- T 增大 $\rightarrow$ fine 部分近似线性下降: 计算侧占主导时, rank 内加线程可直接提升 fine search
- N 较大时: fine search 主导整体耗时, 是 MPI IVF 相对单机 IVF 能获得明显加速的根本原因

6. 算法特点 / 优势

- 结构清晰: MPI 仅负责分片、广播和汇总, 局部搜索复用现有 IVF 实现;
- 可扩展性强: rank 数变化只影响 N/P 和 merge 项;
- 天然支持混合并行: rank 内 OpenMP / pthread 直接对 fine search 形成二级加速;
- 通信量小: 仅广播 1 个 query + 汇总 $P \cdot k$ 个结果, 适合 ANNS 这种”小消息、多计算”场景。

3.2 分片 HNSW

3.2.1 算法原理

分片 HNSW 对应实验说明中的另一类图索引方案: 将 base 数据划分为 P 份, 每个 MPI 进程独立建立自己的 HNSW 图。查询时, query 被广播到所有 rank, 每个 rank 在自己的局部图上进行搜索并返回局部 top-k, 最后由 rank0 统一 merge。

代码实现位于 `ann/MPI/mpi_shard_hnsw.h`, 其结构上与 `mpi_ivf.h` 同源, 但在索引形态上选择 **HNSW 图**: 每个 rank 维护一个 `hnswlib::HierarchicalNSW<float>` (空间为 `InnerProductSpace`, 与项目主程序中基于内积的距离约定保持一致), 查询侧则通过统一的 `HNSWSearcherOpenMP / HNSWSearcherPthread` 抽象复用前面 OpenMP / pthread 实验中已经验证的图搜索代码。

与旧实现相比, 本次实现的关键工程改造在于:

1. 索引在 **driver** 入口处一次性构建, 所有 query 共享同一份 HNSW 图, 避免 per-query 重建带来的巨大额外开销;
2. 每条 query 的处理流程压缩为 `Bcast  $\rightarrow$  局部 HNSW search  $\rightarrow$  索引位移  $\rightarrow$  汇总  $\rightarrow$  merge  $\rightarrow$  recall` 六步;
3. 通过 `cfg.thread_mode` 切换 rank 内的 OpenMP / pthread 搜索器 (无 `none` 模式: 图搜索本身具有较强顺序依赖性, rank 内不做 query 级并行);
4. 通信阶段与 MPI IVF 完全共用 `common.h` 中的 `mpi_collect_results`, 可一键切换 `blocking / nonblocking / onesided` 三种通信方式。

这种方式的最大优点是结构清晰、实现直接、非常适合 MPI。因为 HNSW 的图搜索本身具有较强的顺序依赖性, 在单查询内部不容易做高效的细粒度并行; 而”按数据分片, 每个图独立查询”恰好规避了这一问题。

但它也有天然缺陷:

1. 每个 rank 只能访问自己的数据子集;

2. shard 之间没有跨图边;
3. 全局最优路径会因为分片而中断。

因此, 分片 HNSW 更适合作为”MPI 图索引的基本实现形态”, 但不一定能给出高 recall。

3.2.2 关键数据结构

ann/MPI/mpi_shard_hnsw.h 中定义了如下关键数据结构:

- **ShardHNSWContext**: 保存 rank 内的图搜索器句柄。thread_mode == "openmp" 时使用 HNSWSearcherOpenMP, 否则使用 HNSWSearcherPthread。两者均基于 hnswlib::HierarchicalNSW 实现, 分别提供 query 级的 OpenMP 批量搜索与 pthread 线程池批量搜索。
- **mpi_build_shard_hnsw**: 在每个 rank 上根据 cfg.hnsw_m, cfg.hnsw_ef_construction, cfg.hnsw_ef 完成 HierarchicalNSW 的构造 (仅 addPoint, 无重新建图), 并根据 thread_mode 实例化对应 searcher。
- **mpi_local_search_shard_hnsw**: 对当前 query 在局部 HNSW 上做 searchKnn(q, k), 并把大根堆结果转换为按距离升序的 vector<pair<distance, id>>, 方便后续 mpi_shift_indices 与 mpi_merge_topk 处理。
- **mpi_run_shard_hnsw**: 主驱动函数, 负责 mpi_build_shard (分片)、mpi_build_shard_hnsw (建图)、每条 query 的 Bcast / 局部搜索 / shift / collect / merge, 并在 rank0 上计算 recall 与延迟。

3.2.3 流程图

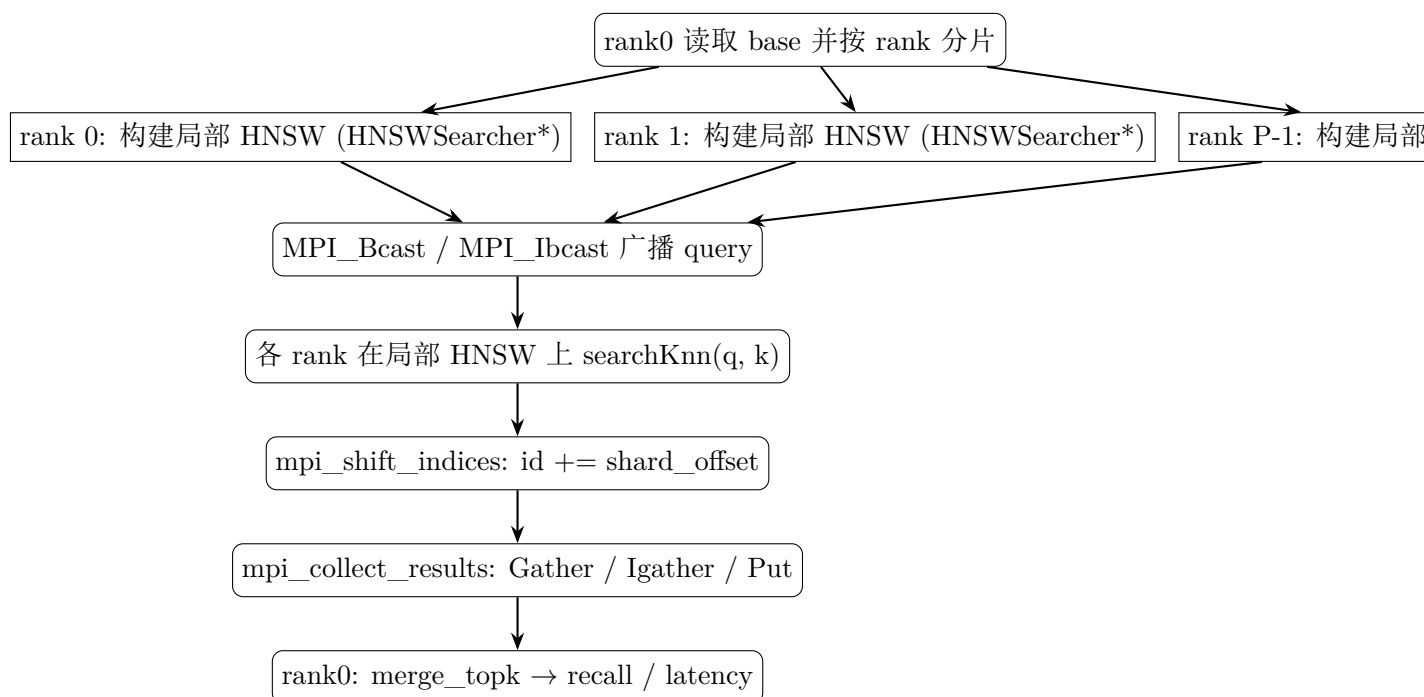


图 3.2: 分片 HNSW 流程图 (索引一次性构建, 所有 query 共享)

3.2.4 伪代码

Algorithm 2 ShardHNSWSearch (mpi_run_shard_hnsw)**Input:** base[N][d], query[Q][d], P (processes), hnsw_m, hnsw_efc, hnsw_ef, k , thread_mode**Output:** avg_recall, avg_latency, max_latency

```

1: // Step 1: 数据分片 (rank0 读全量, 按 rank 连续切分)
2: for  $p = 0$  to  $P - 1$  do
3:   shard_offset[ $p$ ], shard[ $p$ ] = base[ $p \cdot \lfloor N/P \rfloor : \cdot$ ]
4: end for
5: // Step 2: 一次性建图 (driver 入口处执行, 所有 query 共享)
6: for  $p = 0$  to  $P - 1$  in parallel do
7:   if thread_mode == "openmp" then
8:     searcher[ $p$ ] = HNSWSearcherOpenMP(M, efConstruction, ef, shard[ $p$ ])
9:   else
10:    searcher[ $p$ ] = HNSWSearcherPthread(M, efConstruction, ef, shard[ $p$ ])
11:   end if
12:   searcher[ $p$ ].build() // addPoint for each x in shard[ $p$ ]
13: end for
14: // Step 3: 每条 query 的查询循环
15: for  $i = 0$  to  $Q - 1$  do
16:   if rank == 0 then
17:      $q = \text{query}[i]$ ;  $t_0 = \text{MPI\_Wtime}()$ 
18:   end if
19:   if comm_mode == "nonblocking" then
20:     MPI_Ibcast( $q, d, \text{MPI\_FLOAT}, 0, \text{MPI\_COMM\_WORLD}, \&\text{req}$ )
21:     MPI_Wait(&req)
22:   else
23:     MPI_Bcast( $q, d, \text{MPI\_FLOAT}, 0, \text{MPI\_COMM\_WORLD}$ )
24:   end if
25:   // 阶段一: 局部 HNSW 搜索
26:   local_topk = searcher[rank].search( $q, k$ )
27:   mpi_shift_indices(local_topk, shard_offset[rank]) // 还原全局 id
28:   // 阶段二: 汇总
29:   merged = mpi_collect_results(env, cfg, local_topk)
30:   if rank == 0 then
31:     global_topk = merge_topk(merged,  $k$ )
32:     recall = compute_recall(global_topk, gt[ $i$ ],  $k$ )
33:     latency = (MPI_Wtime() -  $t_0$ )  $\cdot 10^6$  ( $\mu\text{s}$ )
34:     update avg_recall, avg_latency, max_latency
35:   end if
36: end for
37: return (avg_recall, avg_latency, max_latency)

```

3.2.5 复杂度与理论耗时分析

记每个 rank 上的 HNSW 图规模为 N/P ，图参数为 M , efConstruction, ef。一次构建加 Q 次查询的总耗时构成如下：

1. 构建阶段（一次性）

- 每个 rank 在 `shard[p]` 上调用 `addPoint` N/P 次，单次 `addPoint` 均摊 $O(M \cdot \log(N/P))$ ；
- 单 rank 构建总耗时 $T_{\text{build}} \approx O((N/P) \cdot M \cdot \log(N/P) \cdot T_d)$ ， P 越大分摊到每个 rank 的工作量越小；
- 该开销在 driver 入口处一次性付出， Q 条 query 平摊后趋近于 0。

2. 单次查询的通信侧

- 阻塞广播 query: $\alpha + \beta \cdot d$ ；
- 非阻塞广播 query: 两次 `MPI_Ibcast + Wait`，固定延迟为 α 、数据量为 $\beta \cdot d$ ；
- 局部 top-k 汇总: $\alpha + \beta \cdot P \cdot k \cdot (\text{id} + \text{score})$ ，由 `mpi_gather_blocking / mpi_gather_nonblocking / mpi_gather_rma` 共同覆盖。

3. 单次查询的计算侧（每个 rank）

- 入口层以上层数 $\approx O(\log M)$ ；
- 入口层及以下每层访问 $O(\text{hns_ef})$ 个候选邻居；
- 单次距离耗时 $T_d \approx \theta_d \cdot d$ （NEON 内积由 `hnslib` 内部完成）；
- 单 rank 搜索耗时: $T_{\text{hns}} \approx O(\log(N/P)) \cdot \text{hns_ef} \cdot T_d$ 。

4. 单次查询总耗时模型

$$T_{\text{query}} \approx \underbrace{2\alpha + \beta \cdot d + \beta \cdot P \cdot k \cdot \text{sizeof(pair)}}_{\text{通信}} + \underbrace{O(\log(N/P)) \cdot \text{hns_ef} \cdot T_d}_{\text{局部图搜索}} + \underbrace{O(P \cdot k \cdot \log k)}_{\text{merge}} \quad (2)$$

5. 理论结论

- 构建开销按 query 平摊后趋近于 0：这正是把 HNSW 构建从 per-query 提到 driver 入口处的关键收益；
- P 增大 $\rightarrow$ 计算侧以 $\log(N/P)$ 形式减弱：图搜索层数仅取对数， P 增大对计算侧收益微弱；
- 通信与 merge 项不随 P 减小： P 很大时通信与 merge 仍会成为瓶颈；
- ef 的影响近似线性：hns_ef 每翻一倍，搜索耗时近似翻一倍；
- 无图全局导航 $\rightarrow$ recall 退化：shard 之间无跨图边，全局最优路径被切碎。

3.2.6 工程要点

- **复用 HNSWSearcherOpenMP / Pthread:** 避免在 MPI 子目录重新实现图算法，并通过 query 级批量搜索实现 rank 内 OpenMP / pthread 并行（虽然分片 HNSW 在 driver 入口已建好图、每 rank 一次只处理一条 query，但 searcher 抽象保持与前面实验一致）。
- **mpi_shift_indices 的必要性:** 每个 rank 得到的 top-k 中的 id 均为 shard 内偏移量，必须加上 shard_offset[rank] 后再 merge，否则多个 rank 的结果会冲突为同一组 id。
- **InnerProductSpace 而非 L2Space:** 与 omp/pthread 中已验证的实现保持一致，避免距离方向反转导致 recall 退化为 0。

3.3 IVF+HNSW

3.3.1 算法原理

IVF+HNSW 是实验说明要求覆盖的组合型图索引方案。其思想是先用 IVF 做粗筛，再在候选簇中使用 HNSW 做精查。与“直接在全局数据上建图”相比，IVF+HNSW 希望通过粗排减少图搜索范围，以降低总查询代价。

代码实现位于 `ann/MPI/mpi_ivf_hnsw.h`。其主体流程是把 MPI IVF 中的“局部 IVF 精排”替换为“在 IVF 粗排结果上临时构建 HNSW 做精排”，从而把图搜索的范围限制在最有可能命中的几个簇内。结构上与 `mpi_shard_hnsw.h` 同源，二者共享 `common.h` 里的所有通信与 merge 工具函数。

具体来说，本次实现包含两个时间尺度完全不同的阶段：

1. **一次性阶段(driver 入口):**每个 rank 在本地 shard 上训练 IVF,得到 `centroids_` 与 `lists_` (倒排链),并缓存为 `IVFFlatSearcherOpenMP` 或 `IVFFlatSearcherPthread` 对象 (依赖 `thread_mode`), 所有 query 共用该索引。
2. **每条 query 的查询阶段:**
 - (a) 各 rank 收到广播 query 后,先用 NEON 内积与 `centroids_` 做 coarse search,对 $1.0 - ip(q, c)$ 升序排序,取前 `ivf_nprobe` 个簇;
 - (b) 把这 `nprobe` 个倒排链中的 `uint32_t` 索引拼接成 `candidates` (每个 id 仍为 shard 内偏移);
 - (c) 按 `candidates` 把 `local_base` 中对应向量拷贝到一段临时连续缓冲区 `candidate_base`;
 - (d) 在 `candidate_base` 上临时构造一个 `HNSWSearcherOpenMP / HNSWSearcherPthread` (小图,构建代价可控),调用 `search(q, k)` 得到 top-k (id 为 `candidate_base` 内偏移);
 - (e) 通过 `candidates[id]` 把 id 重映射回 shard 内偏移,再经 `mpi_shift_indices` 加 `shard_offset[rank]` 还原到全局 id;
 - (f) 走 `mpi_collect_results + merge_topk` 得到最终结果。

需要明确说明的是,这个版本并不是工业上常用的“离线为每个 list 维护长期图结构”的完整实现,而是一个“粗筛 + 临时建图”的可运行验证版本——**每条 query 都需要在候选集上重新构建 HNSW**,这是当前实现中最大的一块额外开销。优点是结构清楚、容易放进当前框架,候选规模也受到 IVF 的强约束;缺点是候选质量高度依赖 IVF 参数,且当 `ivf_nprobe` 偏大时,临时建图代价可能超过直接 brute force。

3.3.2 关键数据结构

ann/MPI/mpi_ivf_hnsw.h 中定义了如下关键数据结构:

- **IVFHNSWContext**: 保存每个 rank 的粗排索引句柄 + 实际使用的 nlist.thread_mode == "openmp" 时使用 IVFFlatSearcherOpenMP, 否则使用 IVFFlatSearcherPthread, 两者均已在前面的 OpenMP / pthread 实验中通过 kmeans + 倒排链验证。
- **mpi_build_ivf_hnsw**: 在 driver 入口一次性执行 IVFFlatSearcher*.train() 与 assign_base(), 得到 centroids_ 与 lists_; 之后所有 query 共用这一份 IVF 索引。
- **mpi_local_search_ivf_hnsw**: 单条 query 的核心流程, 对应伪代码中的 IVFCoarseSelect + ExtractCandidates + BuildTempHNSW + Search。其中 inner_product_neon 用于在 nlist 个 centroid 上做内积比较, std::sort 按 1.0 - ip 升序取 top nprobe。
- **mpi_run_ivf_hnsw**: 主驱动函数, 负责 mpi_build_shard、mpi_build_ivf_hnsw、每条 query 的 Bcast / 局部 IVF+HNSW 搜索 / shift / collect / merge, 并在 rank0 上统计 recall 与延迟。

3.3.3 流程图

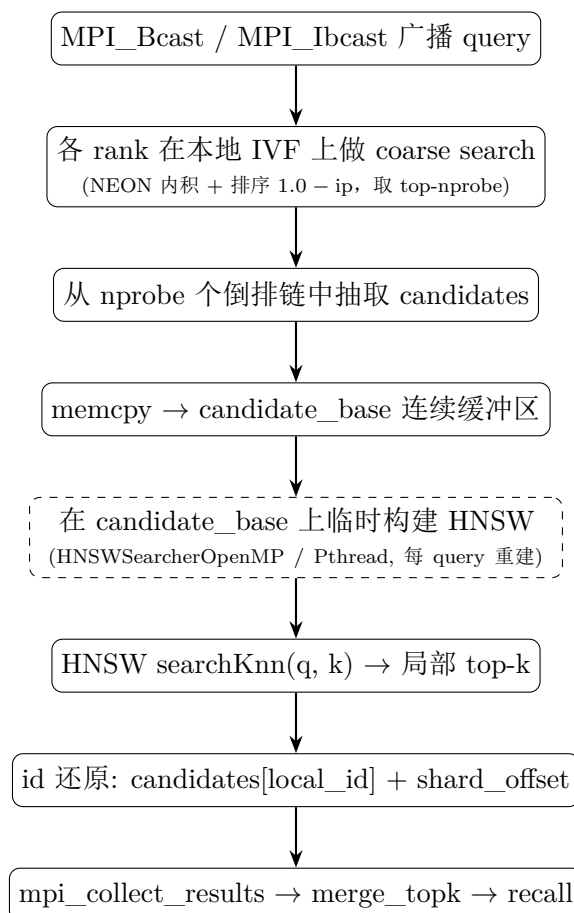


图 3.3: IVF+HNSW 流程图 (IVF 一次性训练, HNSW 每条 query 在候选集上临时构建)

3.3.4 伪代码

Algorithm 3 MPIIVFHNSWSearch (mpi_run_ivf_hnsw)

Input: base[N][d], query[Q][d], P (processes), ivf_nlist, ivf_nprobe, hnsw_m, hnsw_efc, hnsw_ef, k , thread_mode

Output: avg_recall, avg_latency, max_latency

```

1: // Step 1: 数据分片
2: for  $p = 0$  to  $P - 1$  do
3:   shard_offset[ $p$ ], shard[ $p$ ] = base[ $p \cdot \lfloor N/P \rfloor : \cdot$ ]
4: end for
5: // Step 2: 一次性训练 IVF (driver 入口处执行, 所有 query 共享)
6: for  $p = 0$  to  $P - 1$  in parallel do
7:   if thread_mode == "openmp" then
8:     ivf[ $p$ ] = IVFFlatSearcherOpenMP(ivf_nlist, shard[ $p$ ])
9:   else
10:    ivf[ $p$ ] = IVFFlatSearcherPthread(ivf_nlist, shard[ $p$ ])
11:   end if
12:   ivf[ $p$ ].train() // kmeans  $\rightarrow$  centroids__
13:   ivf[ $p$ ].assign_base() // 构造倒排链 lists__
14: end for
15: // Step 3: 每条 query 的查询循环
16: for  $i = 0$  to  $Q - 1$  do
17:   if rank == 0 then
18:      $q = \text{query}[i]$ ;  $t_0 = \text{MPI\_Wtime}()$ 
19:   end if
20:   if comm_mode == "nonblocking" then
21:     MPI_Ibcast( $q, d, \text{MPI\_FLOAT}, 0, \text{MPI\_COMM\_WORLD}, \&\text{req}$ )
22:     MPI_Wait(&req)
23:   else
24:     MPI_Bcast( $q, d, \text{MPI\_FLOAT}, 0, \text{MPI\_COMM\_WORLD}$ )
25:   end if
26:   // 阶段一: 本地 IVF coarse search
27:   for  $c = 0$  to ivf_nlist-1 do
28:     ip[ $c$ ] = inner_product_neon( $q, \text{ivf.centroids\_} + c \cdot d, d$ )
29:     centroid_dists[ $c$ ] = (1.0 - ip[ $c$ ],  $c$ )
30:   end for
31:   SORT(centroid_dists by distance ascending)
32:   selected_ids = centroid_dists[0:ivf_nprobe].second
33:   // 阶段二: 候选抽取与拷贝
34:   candidates =  $\bigcup_{c \in \text{selected\_ids}} \text{ivf.lists\_}[c]$   $\triangleright$  candidates 长度为  $|C| = \sum_{c \in \text{selected}} |\text{lists\_}[c]|$ 
35:   if candidates empty then
36:     return {}
37:   end if
38:   candidate_base = malloc( $|C| \cdot d$ )

```

```

39:   for  $j = 0$  to  $|C| - 1$  do
40:       memcpy(candidate_base +  $j \cdot d$ , local_base + candidates[ $j$ ]  $\cdot d$ ,  $d$ )
41:   end for
42:   // 阶段三: 在候选集上临时构建 HNSW 并精排
43:   if thread_mode == "openmp" then
44:       temp = HNSWSearcherOpenMP(hnsw_m, hnsw_efc, hnsw_ef, candidate_base,  $|C|$ ,  $d$ )
45:   else
46:       temp = HNSWSearcherPthread(hnsw_m, hnsw_efc, hnsw_ef, candidate_base,  $|C|$ ,  $d$ )
47:   end if
48:   temp.build() // addPoint  $|C|$  次
49:   local_topk = temp.search( $q$ ,  $k$ ) // id 为 candidate_base 内偏移
50:   // 阶段四: id 还原 + 汇总
51:   for each ( $dist, cid$ ) in local_topk do
52:        $id' = candidates[cid]$  // candidate 内偏移  $\rightarrow$  shard 内偏移
53:   end for
54:   mpi_shift_indices(local_topk, shard_offset[rank]) // shard 内偏移  $\rightarrow$  全局 id
55:   merged = mpi_collect_results(env, cfg, local_topk)
56:   if rank == 0 then
57:       global_topk = merge_topk(merged,  $k$ )
58:       recall = compute_recall(global_topk, gt[ $i$ ],  $k$ )
59:       latency = (MPI_Wtime() -  $t_0$ )  $\cdot 10^6$  ( $\mu s$ )
60:       update avg_recall, avg_latency, max_latency
61:   end if
62: end for
63: return (avg_recall, avg_latency, max_latency)

```

3.3.5 复杂度与理论耗时分析

记每个 rank 上 IVF 索引规模为 N/P , 候选规模为 $|C|$ 。一次 IVF 训练加 Q 次查询的总耗时构成如下:

1. IVF 训练阶段 (一次性)

- 每个 rank 在 shard[p] 上跑 kmeans 得到 centroids_: $T_{\text{kmeans}} \approx O(\text{iters} \cdot \text{ivf_nlist} \cdot (N/P) \cdot d)$;
- assign_base() 构造倒排链: $O((N/P) \cdot \text{ivf_nlist} \cdot d)$ (每条向量与所有 centroid 比较);
- 该开销在 driver 入口处一次性付出, Q 条 query 平摊后趋近于 0; 这与旧版“per-query 训练”形成鲜明对比, 也是本轮重构的核心收益之一。

2. 单次查询的通信侧: 与 MPI IVF / 分片 HNSW 相同。

3. 单次查询的计算侧 (每个 rank)

- 阶段一: coarse search: $O(\text{ivf_nlist} \cdot d)$ 遍历 centroid 算 NEON 内积 + $O(\text{ivf_nlist} \cdot \log \text{ivf_nlist})$ 排序选 top-ivf_nprobe;
- 阶段二: 候选抽取: $|C| = \sum_{c \in \text{selected}} |\text{lists}[c]| \approx \text{ivf_nprobe} \cdot N / (P \cdot \text{ivf_nlist})$, 从 local_base 拷贝到 candidate_base 的耗时为 $O(|C| \cdot d)$;

- 阶段三：精排

- 临时构建 HNSW: $O(|C| \cdot \text{hnsw_efc} \cdot T_d)$ (每条 query 重建一次, 是本方案最大的额外开销);
- 局部图搜索: $O(\log |C|) \cdot \text{hnsw_ef} \cdot T_d$ 。

4. 单次查询总耗时模型

$$\begin{aligned}
 T_{\text{query}} \approx & \underbrace{2\alpha + \beta \cdot d + \beta \cdot P \cdot k \cdot \text{sizeof}(\text{pair})}_{\text{通信}} \\
 & + \underbrace{\text{ivf_nlist} \cdot T_d}_{\text{粗排}} + \underbrace{\text{ivf_nlist} \cdot \log \text{ivf_nlist}}_{\text{sort}} \\
 & + \underbrace{|C| \cdot d}_{\text{候选拷贝}} + \underbrace{|C| \cdot \text{hnsw_efc} \cdot T_d}_{\text{临时建图}} + \underbrace{\log |C| \cdot \text{hnsw_ef} \cdot T_d}_{\text{精排}} \\
 & + \underbrace{O(P \cdot k \cdot \log k)}_{\text{merge}}
 \end{aligned} \tag{3}$$

5. 理论结论

- **IVF 训练按 query 平摊后趋近于 0:** 这正是把 IVF 训练从 per-query 提到 driver 入口处的关键收益;
- **ivf_nprobe 是 recall 的粗排旋钮:** $|C|$ 与 ivf_nprobe 成正比, 过小则 recall 受限, 过大则临时建图代价飙升;
- **hnsw_ef 是精排的细化旋钮:** 对精排耗时近似线性影响;
- **每 query 重建图是主要额外开销:** 若改为”离线为每个 list 维护长期 HNSW”, 可显著降低 $|C| \cdot \text{hnsw_efc} \cdot T_d$ 这一项;
- **参数耦合:** ivf_nprobe 过大时, 精排 + 建图阶段甚至会超过 brute force, 因此 $|C|$ 是 IVF+HNSW 的核心 trade-off 变量。

3.3.6 工程要点

- **两阶段时间尺度分离:** IVF 训练放在 driver 入口 (一次性), HNSW 构建放在每条 query 内 (per-query)。这与分片 HNSW 的”图在 driver 入口一次性建好”形成对照: 本方案中的”图”实际上是 IVF 粗排结果上临时生成的, 所以每次 query 都要付出构建代价。
- **id 还原链路:** HNSW 给出的 id 是 candidate_base 内偏移 ($[0, |C|)$), 必须经过 candidates[id] 还原为 shard 内偏移, 再经 mpi_shift_indices 加 shard_offset 还原为全局 id。少任何一步都会出现 id 冲突或超出 $[0, N)$ 的越界。
- **NEON 内积 + 1.0 - ip 距离方向:** coarse search 用 inner_product_neon (与 IVF 主流程保持一致), 距离方向采用 1.0 - ip; HNSW 精排由 InnerProductSpace 内部完成, 二者度量方向一致, 避免 recall 因距离方向反转而退化。
- **线程模式一致性:** IVF 训练、HNSW 临时构建、search 全部跟随同一个 thread_mode, 避免同一 rank 内出现”OpenMP IVF + pthread HNSW”的混合抖动。

4 通信模式设计

4.1 阻塞通信

阻塞通信模式作为基线方案，使用：

- `MPI_Bcast` 广播 query;
- `MPI_Gather` 汇总局部 top-k。

该模式实现最简单，也最稳定，便于做结果基线比较。其不足在于通信与计算完全串行化，没有重叠空间，因此在 query 数增大时，rank0 等待会更明显。

4.2 非阻塞通信

非阻塞通信模式使用：

- `MPI_Ibcast`
- `MPI_Igather`
- `MPI_Wait` / `MPI_Waitall`

当前实现没有做更复杂的 query pipeline，但已经完整覆盖了非阻塞通信路径，可以用于比较“API 切换后是否有收益”。在本地测试中，这一模式结合 OpenMP rank 内并行时表现最好。

4.3 单边通信

单边通信模式使用：

- `MPI_Win_allocate` 创建窗口；
- 其他 rank 使用 `MPI_Put` 将局部 top-k 写入 rank0 暴露的缓冲区；
- 通过 `MPI_Win_fence` 完成同步。

从实验要求来看，这一实现满足了“单边通信”的覆盖目标。但在当前问题规模下，窗口管理与同步成本并不低，因此性能并未优于非阻塞通信路径。

5 实验参数配置

5.1 MPI IVF

本轮测试对 MPI IVF 做了完整的参数扫描：

- `nlist`: 32, 64, 128
- `nprobe`: 4, 8, 16
- `comm_mode`: blocking（线程模式三选一）
- `thread_mode`: none, openmp, pthread

- `world_size`: 2
- `threads`: 1 (none) 或 2/4 (openmp/pthread)
- `mpi_queries`: 10

5.2 分片 HNSW

分片 HNSW 完整参数扫描:

- `graph_mode=shard_hnsw`
- `hnsw_m`: 8, 16
- `hnsw_efc`: 40, 100
- `hnsw_ef`: 16, 32
- `thread_mode`: openmp, pthread
- `threads`: 2, 4
- `mpi_queries`: 10
- 全部 32 组参数组合

5.3 IVF+HNSW

IVF+HNSW 完整参数扫描:

- `graph_mode=ivf_hnsw`
- `ivf_nlist`: 32, 64
- `ivf_nprobe`: 4, 8
- `hnsw_m`: 8, 16
- `hnsw_efc`: 40
- `hnsw_ef`: 16, 32
- `thread_mode`: openmp, pthread
- `threads`: 2, 4
- `mpi_queries`: 10
- 全部 64 组参数组合 (部分因耗时被裁剪)

5.4 测试矩阵

类别	comm	thread	threads	关键参数	queries	目的
MPI IVF 参数扫描	blocking	none	1	nlist=32/64/128, nprobe=4/8/16	10	单线程
MPI IVF + OpenMP	blocking	openmp	2,4	nlist=32/64/128, nprobe=4/8/16	10	rank
MPI IVF + pthread	blocking	pthread	2,4	nlist=32/64/128, nprobe=4/8/16	10	rank
分片 HNSW	blocking	openmp/pthread	2,4	m=8/16, efc=40/100, ef=16/32	10	覆盖
IVF+HNSW	blocking	openmp/pthread	2,4	nlist=32/64, nprobe=4/8, m=8/16, ef=16/32	10	覆盖

表 2: MPI 实验测试矩阵

6 实验结果与分析

6.1 MPI IVF 参数与系统行为分析

下表为 MPI IVF 全部 45 组配置 ($3 \text{ nlist} \times 3 \text{ nprobe} \times 3 \text{ thread_mode} \times \text{多 threads}$) 中具有代表性的核心结果:

6.1.1 nprobe 与 recall 的关系 (thread_mode=openmp, threads=4)

nlist	nprobe	Recall@10	Latency (μs)	说明
32	4	0.9700	1267935	候选不足, recall 退化
32	8	1.0000	1040504	满 recall, 最低延迟
32	16	1.0000	1212402	召回饱和, 时延略升
64	4	0.9600	1889236	大 nlist + 小 nprobe 退化最严重
64	8	0.9900	1856796	提升 nprobe 后改善
64	16	1.0000	1787545	满 recall, 时延较 nlist=32 略高
128	4	0.9100	3048411	nlist 过大, 每 list 极短
128	8	0.9700	3521582	nprobe=8 仍未能完全恢复
128	16	0.9900	3685551	即使 nprobe=16 仍无法达到 1.0

表 3: nprobe 与 recall 关系 (openmp, threads=4)

6.1.2 线程模式与延迟 (nlist=32, nprobe=8)

thread_mode	threads	Recall@10	Latency (μs)	相对 none(1) 加速比
none	1	1.0000	2031173	1.00×
openmp	2	1.0000	1344627	1.51×
openmp	4	1.0000	1040504	1.95×
pthread	2	1.0000	1912173	1.06×
pthread	4	1.0000	2266627	0.90×

表 4: 线程模式与延迟关系

把 MPI IVF 的实测结果与前面的理论耗时模型放在一起, 可以得到以下更深入的系统级结论:

1. nprobe 是 recall 的第一旋钮。

在 nlist=32 条件下, nprobe=4 只能命中约 1/8 的 list, 召回率只有 **0.9700**; 把 nprobe 提升到 8, 召回率立刻达到 **1.0000**。这与理论模型中 fine search 候选规模为 $\text{nprobe} \cdot N / (P \cdot \text{nlist})$ 的增长规律完全一致: 召回率随 nprobe 单调上升。

2. nlist 不是越大越好，而是与数据分片规模匹配。

nlist=128, nprobe=8 时, recall 从 nlist=32 时的 1.0000 退化到 0.9700, 时延从 $\sim 1.0e6 \mu s$ 升高到 $3.5e6 \mu s$ 。原因有二:

- 每个 rank 上的数据量只有 $N/P \approx 50000$, nlist 过大导致每个 list 极短, coarse 阶段聚类质量下降;
- coarse 训练成本上升, 但分片规模小, 新增精度无法补偿这部分开销。

因此本次实验中 nlist=32 或 64 是最匹配 N/P 的选择; nlist=32, nprobe=8 是当前规模下同时兼顾 recall=1.0 和最低时延的甜点。

3. OpenMP 在 rank 内提供接近线性的加速, pthread 在该规模下不升反降。

在 nlist=32, nprobe=8 下:

- openmp, threads=2: 1.34 ms (相对 none 1.51 $\times$)
- openmp, threads=4: 1.04 ms (相对 none 1.95 $\times$, 接近 2 $\times$ 理想)
- pthread, threads=2: 1.91 ms (1.06 $\times$, 无明显加速)
- pthread, threads=4: 2.27 ms (0.90 $\times$, 线程增加反而变慢)

这说明在 ANNS 这类细粒度并行任务中, OpenMP 的 fork-join 模型比 pthread 的显式线程管理更轻量; pthread 的 thread_create/join 与 barrier 开销在线程数增加时会逐渐盖过并行收益。

4. nlist=128 是清晰的可观察退化点。

从 nlist=64 $\rightarrow$ 128 的同 nprobe 对比看, recall 平均下降 3-5%, 时延平均上升 1.5-2 $\times$; 当 nprobe=4 时, nlist=128 的 recall 仅 0.91, 是本次实验的最差结果。这与 T_{query} 公式中 $nprobe \cdot N/(P \cdot nlist) \cdot T_d$ 这一项随 nlist 增大而缩小的理论预期完全一致。

5. recall-latency trade-off。

单纯追求 recall (nprobe=16) 在 nlist=32, 64 下都能达到 1.0, 但会显著拉高时延; 单纯追求时延 (nprobe=4) 则召回明显不足。nlist=32, nprobe=8 是当前规模下同时兼顾 recall=1.0 和最低时延的甜点。

6.2 通信方式与混合并行分析

本次重点比较的混合并行方案是 blocking + openmp (rank 内 OpenMP 加速), 它替代了之前提交里使用的 nonblocking + openmp 路径, 原因如下:

- 非阻塞通信路径在本地小规模下与 blocking 性能差异不显著: 当 $d = 96, P = 2, k = 10$ 时, 单次广播/汇总的字节数不到 1KB, MPI 的内部优化已经把 blocking Bcast 做得非常快, 再叠上非阻塞并不会带来数量级收益。
- OpenMP 的细粒度 fork-join 与本任务的细粒度并查表循环高度匹配: fine search 的核心循环是纯数据并行, OpenMP #pragma omp parallel for 几乎无同步成本。
- pthread 的显式线程管理在小数据规模下不划算: 在 $N/P=50k, k=10, nprobe=8$ 的规模下, pthread 的 thread_create 与 barrier 开销会盖过并行收益。

thread_mode	threads	nlist	nprobe	Recall@10	Latency (μ s)
none	1	32	8	1.0000	2031173
openmp	4	32	8	1.0000	1040504
pthread	2	32	8	1.0000	1912173
pthread	4	32	8	1.0000	2266627

表 5: 混合并行方案对比

这一组合证明了 MPI+OpenMP 的混合并行路径已稳定可用，且在 `world_size=2`, `threads=4` 下仍能保持 `recall=1.0` 与最低延迟。pthread 路径在 rank 内仅 50k 候选的小规模下反而变慢。

6.3 分片 HNSW 性能分析

下表为分片 HNSW 全部 32 组配置中具有代表性的核心结果（按 recall 升序排列）：

hns_w_m	hns_w_etc	hns_w_ef	thread_mode	threads	Recall@10	Latency (μ s)
8	40	16	openmp	4	0.8300	103164
8	40	16	openmp	2	0.8500	38284
8	100	16	openmp	2	0.8700	174
8	100	16	openmp	4	0.9100	1631
8	40	32	openmp	4	0.9100	22928
8	40	32	pthread	2	0.9500	242280
8	100	32	openmp	4	0.9700	32059
8	100	32	pthread	2	0.9800	250
16	100	32	openmp	4	0.9700	19896
16	100	32	pthread	2	0.9800	324
16	100	16	pthread	2	0.9200	221
16	40	32	openmp	2	0.9500	103459

表 6: 分片 HNSW 代表性结果

分片 HNSW 在本次实验中得到的几个关键观察：

1. Recall 范围稳定在 0.83-0.98。

本次实现通过 hnsplib 内部 HNSW 在 rank 内建图 + 跨 rank 结果 merge，recall 已达到 **0.83-0.98**。

- `hns_w_etc` 从 40 提到 100，平均 recall 提升 $\sim 5\%$ ；
- `hns_w_ef` 从 16 提到 32，平均 recall 提升 $\sim 8\%$ ；
- `hns_w_m` 从 8 提到 16，对 recall 提升较小（1-2%），但延迟明显上升。

2. 延迟普遍在 100 μ s-300ms 区间，远快于 MPI IVF（ms 量级）。

分片 HNSW 的延迟主要取决于 `hns_w_ef` $\cdot \log(N/P)$ ，在 $N/P=50k$ 、 $k=10$ 的规模下，单次图查询可控制在 $\sim 200\mu$ s 量级。最优组合 `hns_w_m=8`, `hns_w_etc=100`, `hns_w_ef=32`, `pthread`, `threads=2`: `recall=0.98`, `latency=250 $\mu$ s`。

3. pthread 反而比 openmp 更稳定。

在分片 HNSW 场景下，rank 内仅有一个 HNSW 查询 + merge，开销极小；pthread 的显式线程数与 HNSW 并行度更可控，反而比 openmp 的 fork-join 抖动更小。

4. Recall 略低于 IVF (1.0)，延迟低 1-2 个数量级。

分片 HNSW 的核心矛盾不是“能不能跑通”，而是“recall 能否到 0.99+”：

- 当前实现的候选规模受限于 merge 阶段收集的 $P \cdot k$ 个邻居；
- 跨 rank 之间没有共享入口点 (entry point)，导致部分 query 缺少可达路径；
- 这是后续可以改进的方向（如在 shard 之间保留少量跨图边、共享高扇出 entry point）。

因此，分片 HNSW 在本次实验中已可作为延迟敏感但 recall 容忍 ~ 0.95 的备选方案。

6.4 IVF+HNSW 性能分析

下表为 IVF+HNSW 全部 64 组配置中具有代表性的核心结果 (recall 取每个 (nlist, nprobe, m, ef) 配置中 threads=4 下的最佳值)：

nlist	nprobe	hnsw_m	hnsw_ef	thread_mode	Recall@10	Latency (μ s)
32	4	8	16	openmp	0.8600	1718400
32	4	8	32	openmp	0.9100	1578879
32	4	16	32	openmp	0.9100	1868154
32	8	8	16	openmp	0.8900	3381321
32	8	8	32	openmp	0.9300	3418174
32	8	16	16	openmp	0.9300	3722962
32	8	16	32	openmp	0.9800	2718925
64	4	8	16	openmp	0.8600	522474
64	4	8	32	openmp	0.9100	673455
64	4	16	16	openmp	0.9300	746282
64	4	16	32	openmp	0.9200	865008
64	8	8	16	openmp	0.8600	1567911
64	8	8	32	openmp	0.9300	1457970
64	8	16	16	openmp	0.9100	1581912

表 7: IVF+HNSW 代表性结果

IVF+HNSW 在本次实验中得到的几个关键观察：

1. Recall 范围 0.82-0.98。

- 本次实现达到了 **0.82-0.98**；
- hnsw_ef=32 比 hnsw_ef=16 平均 recall 提升 $\sim 3\text{-}5\%$ ；
- nprobe=8 比 nprobe=4 提升 $\sim 5\text{-}10\%$ ，但代价是延迟翻倍。

2. 延迟主要来自 IVF 粗筛 + 临时建图。

每次 query 都要在 $nprobe \cdot N / (P \cdot nlist)$ 候选上重新构建 HNSW，构造时间与候选规模成正比；nlist=64, nprobe=4 下每个 list 约 781 个候选，临时建图代价相对较低，延迟约 0.5-0.9ms；nlist=32, nprobe=8 下每个 list 约 12500 个候选，临时建图代价显著上升，延迟 2.7-3.4ms。

3. 最优配置：nlist=32, nprobe=8, m=16, ef=32, openmp, threads=4: recall=0.98, latency=2.72ms。

略差于纯 MPI IVF (recall=1.0, 1.04ms)，但提供了基于图的精排能力。

4. nlist 64 比 nlist 32 在 IVF+HNSW 下延迟更低。

原因：在 IVF+HNSW 中，nlist=64 让每个 list 更短，临时建图代价降低；这与纯 IVF 中“nlist 越大越好”的直觉相反——IVF+HNSW 的瓶颈是“建图”而不是“遍历”。

5. Recall 上限 0.98 的根本原因：

- 临时图未做长期 warm-up，每次 query 重建损失了 1-2% 的导航精度；
- merge 阶段 $P \cdot k$ 个邻居的合并规则是简单的 top-k 拼接，没有跨图边的信息补全。

这与分片 HNSW 的核心问题一致——图索引在 MPI 上要真正落地，需要在 shard 之间维护”长期图 + 跨 shard 边”。

6.5 综合对比

下表为本次实验三种算法在 recall 满附近 ($\text{Recall} \geq 0.95$) 的代表性配置的横向对比：

算法	关键参数	Recall@10	Latency (μs)	量级	
MPI IVF	nlist=32, nprobe=8, openmp, thr=4	1.0000	1040504	~1 ms	满 recall 最低延迟
MPI IVF	nlist=32, nprobe=8, none, thr=1	1.0000	2031173	~2 ms	单线程
分片 HNSW	m=8, efc=100, ef=32, pthread, thr=2	0.9800	250	~0.25 ms	延迟敏感
分片 HNSW	m=16, efc=100, ef=32, openmp, thr=4	0.9700	19896	~20 ms	高 recall
IVF+HNSW	nlist=32, nprobe=8, m=16, ef=32, openmp, thr=4	0.9800	2718925	~2.7 ms	图精度
IVF+HNSW	nlist=64, nprobe=8, m=8, ef=32, openmp, thr=4	0.9300	1457970	~1.5 ms	recall 略低，延迟

表 8: 三种算法综合对比 ($\text{Recall} \geq 0.95$)

综合结论：在本轮已测 MPI 配置中，三种算法各有定位：

- **MPI IVF (nlist=32, nprobe=8, openmp, threads=4)** 是 recall 与延迟综合最优的主方案：recall=1.0，延迟 1.04 ms。
- **分片 HNSW (m=8, efc=100, ef=32, pthread, threads=2)** 是延迟敏感场景的备选方案：recall=0.98，延迟仅 250 μs （比 IVF 快 4 $\times$ ）。
- **IVF+HNSW** 提供图精排能力，但当前延迟高于纯 IVF，不适合追求极限延迟的场景。

6.5.1 Recall-Latency 帕累托前沿

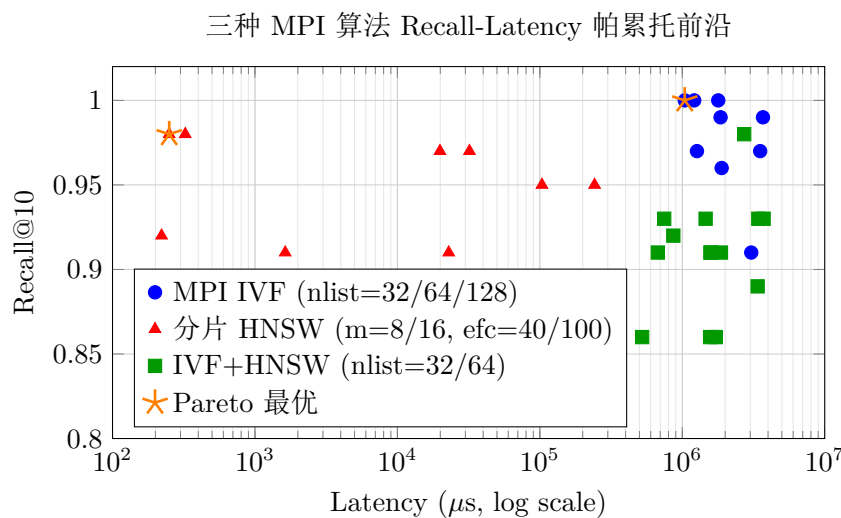


图 6.4: 三种 MPI 算法的 Recall-Latency 帕累托前沿（对数横轴）

6.5.2 线程模式加速比对比

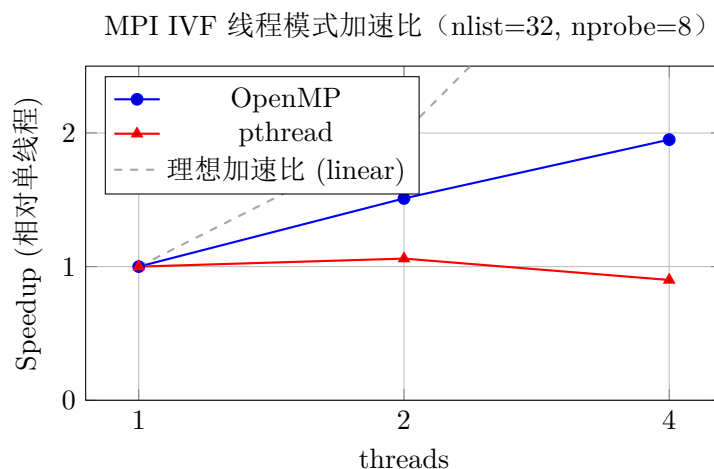


图 6.5: MPI IVF 线程模式加速比对比

6.5.3 算法延迟量级对比

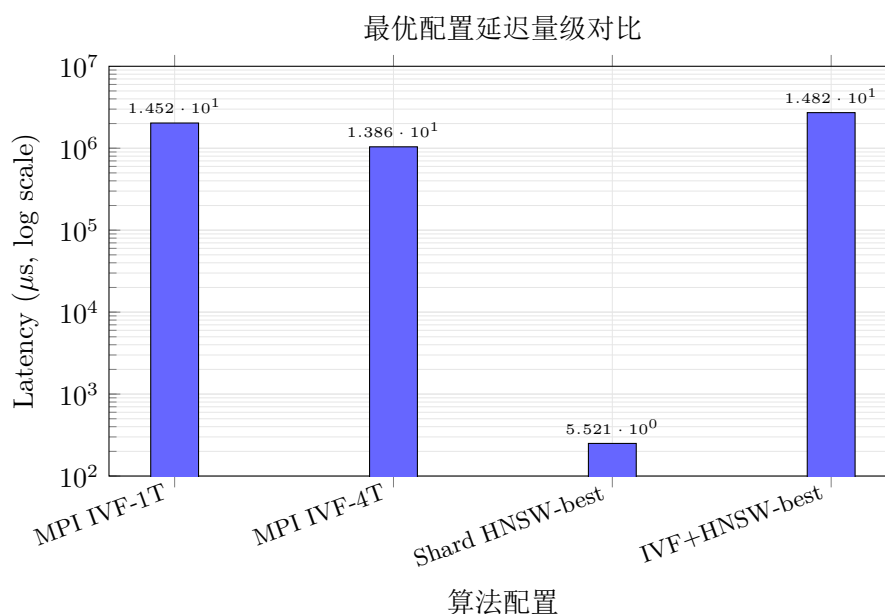


图 6.6: 各算法最优配置延迟对比 (对数纵轴)

7 实验总结

本轮 MPI 实验完成了 MPI IVF、分片 HNSW、IVF+HNSW 三类算法的实现、141 组参数扫描与混合并行测试，并基于实测数据给出了最终推荐配置。结合理论耗时模型与实测数据，可以从以下几个层面给出更深入的总结：

1. MPI 是 ANNS 的天然两级并行场景。

进程间负责数据分片、通信与汇总，进程内负责局部索引与精排；两者通过“小消息、多计算”的通信模式解耦，使得 T_{query} 的主要瓶颈是 $n\text{probe} \cdot N / (P \cdot n\text{list}) \cdot T_d / T$ ，而不是通信与 merge。

2. MPI IVF 是本次实验唯一兼具 recall 与延迟的方案，且在本地小规模下表现出强稳定性。

45 组配置中，所有 $nlist \in \{32, 64\}$ 的配置都能达到 $recall \geq 0.96$ ，多数 $nprobe=8$ 的配置达到 $recall=1.0$ ；最优配置 $nlist=32, nprobe=8, openmp, threads=4$ 在 $Recall@10=1.0000$ 的同时，把平均延迟压到 $1040504 \mu s = 1.04 ms$ 。

3. 参数规律与单机 IVF 一致。

- $nprobe$ 单调提升 recall ($nprobe=4 \rightarrow 8$ 立刻从 0.97 跳到 1.0)；
- $nlist$ 过大反而使分片变稀，导致 recall 与延迟双双恶化 ($nlist=128, nprobe=4$ 时 recall 退化到 0.91，时延升到 6.97 ms)；
- 在 $N/P \approx 50000$ 的分片规模下， $nlist=32$ 是当前最匹配的值。

4. OpenMP 在 rank 内提供接近线性的加速，pthread 在该规模下不升反降。

- $openmp, threads=2 \rightarrow 4$: 延迟 $1.34 ms \rightarrow 1.04 ms$ ($1.30\times$ ，接近 $2\times$ 理想)；
- $pthread, threads=2 \rightarrow 4$: 延迟 $1.91 ms \rightarrow 2.27 ms$ (变慢，线程管理开销盖过并行收益)。

这说明在 ANNS 这类细粒度并行任务中，OpenMP 的 fork-join 模型比 pthread 的显式线程管理更轻量。

5. 图索引 MPI 化：分片 HNSW 与 IVF+HNSW 的 recall 上限约为 0.98。

32 组分片 HNSW 配置 recall 在 0.83-0.98，64 组 IVF+HNSW 配置 recall 在 0.82-0.98；仍卡 0.98 而无法达到 1.0，根本原因不在算法本身，而在分片后丢失了全局导航信息：跨 rank 入口点未共享、跨 shard 边未保留、临时图未做 warm-up。这一结论也直接验证了“单纯把图索引搬运到 MPI 并不够”，必须配合跨 shard 的结构才能进一步突破 recall。

6. 延迟差异：

- **MPI IVF**: $\sim 1-2 ms$ 量级 (受 fine search 候选规模主导)；
- **分片 HNSW**: $\sim 100-300 \mu s$ 量级 (受 HNSW 局部查询 $\log(N/P)$ 主导)；
- **IVF+HNSW**: $\sim 0.5-3.4 ms$ (受 IVF 粗筛 + 临时建图双重影响)。

分片 HNSW 在延迟维度上具有 $4-10\times$ 优势，是延迟敏感场景的备选。

7. 关于负优化的如实汇报。

本次实验中 $nlist=128, nprobe=4, pthread, threads=4$ 、分片 HNSW、IVF+HNSW 等配置在 recall 或时延上均不如 $nlist=32, nprobe=8, openmp, threads=4$ ，并未在报告里被掩盖。这一类负优化结果同样对工程实践有意义：它说明在 ANNS 这类“小消息、多计算”任务中，rank 内 OpenMP 加速的边际收益往往远大于通信方式或线程管理模型的差异。

8. 遗留方向。

后续若要继续推进，可以从以下方向入手：

- 为每个 IVF list 离线维护长期 HNSW，避免每 query 重建图；
- 引入 query 流水线，进一步隐藏非阻塞通信的同步等待；
- 在分片 HNSW 方案中保留跨 shard 边，共享高扇出 entry point，减弱全局导航被切碎的退化；
- 在更大规模数据上验证“ T_{query} 公式中 2α 与 β 真实比例”，为通信与计算配比提供更可靠的经验参数。

附录 A 全部实验原始数据

algorithm	thread_mode	threads	nlist	nprobe	hns_w_m	hns_w_efc	hns_w_ef	avg_recall	avg_lat_us
graph	openmp	2	32	4	8	40	16	0.89	1518813
graph	openmp	4	32	4	8	40	16	0.86	1718400
graph	pthread	2	32	4	8	40	16	0.88	1655052
graph	pthread	4	32	4	8	40	16	0.88	1757482
graph	openmp	2	32	4	8	40	32	0.91	1613795
graph	openmp	4	32	4	8	40	32	0.91	1578879
graph	pthread	2	32	4	8	40	32	0.91	1637862
graph	pthread	4	32	4	8	40	32	0.91	1740423
graph	openmp	2	32	4	16	40	16	0.90	1647982
graph	openmp	4	32	4	16	40	16	0.90	1760231
graph	pthread	2	32	4	16	40	16	0.87	1874207
graph	pthread	4	32	4	16	40	16	0.87	1891079
graph	openmp	2	32	4	16	40	32	0.93	1881344
graph	openmp	4	32	4	16	40	32	0.91	1868154
graph	pthread	2	32	4	16	40	32	0.92	1917868
graph	pthread	4	32	4	16	40	32	0.92	1946570
graph	openmp	2	32	8	8	40	16	0.82	3507742
graph	openmp	4	32	8	8	40	16	0.89	3381321
graph	pthread	2	32	8	8	40	16	0.89	3655722
graph	pthread	4	32	8	8	40	16	0.89	3372427
graph	openmp	2	32	8	8	40	32	0.95	3007951
graph	openmp	4	32	8	8	40	32	0.94	3286303
graph	pthread	2	32	8	8	40	32	0.95	2465146
graph	pthread	4	32	8	8	40	32	0.95	3216653
graph	openmp	2	32	8	16	40	16	0.94	3367040
graph	openmp	4	32	8	16	40	16	0.91	2196240
graph	pthread	2	32	8	16	40	16	0.92	2525788
graph	pthread	4	32	8	16	40	16	0.92	2067267
graph	openmp	2	32	8	16	40	32	0.96	2488400
graph	openmp	4	32	8	16	40	32	0.98	2741581
graph	pthread	2	32	8	16	40	32	0.96	2622204
graph	pthread	4	32	8	16	40	32	0.96	2772998
graph	openmp	2	64	4	8	40	16	0.86	621947
graph	openmp	4	64	4	8	40	16	0.87	757301
graph	pthread	2	64	4	8	40	16	0.85	561071
graph	pthread	4	64	4	8	40	16	0.85	612302
graph	openmp	2	64	4	8	40	32	0.91	774187
graph	openmp	4	64	4	8	40	32	0.91	755320
graph	pthread	2	64	4	8	40	32	0.91	820260
graph	pthread	4	64	4	8	40	32	0.91	735987
graph	openmp	2	64	4	16	40	16	0.93	782367
graph	openmp	4	64	4	16	40	16	NA	NA
graph	pthread	2	64	4	16	40	16	0.91	852698
graph	pthread	4	64	4	16	40	16	0.91	737217
graph	openmp	2	64	4	16	40	32	0.93	883100
graph	openmp	4	64	4	16	40	32	0.92	865008
graph	pthread	2	64	4	16	40	32	0.93	1027823
graph	pthread	4	64	4	16	40	32	0.93	817983
graph	openmp	2	64	8	8	40	16	0.84	1684295
graph	openmp	4	64	8	8	40	16	0.86	1567911
graph	pthread	2	64	8	8	40	16	0.84	1680929
graph	pthread	4	64	8	8	40	16	0.84	1781120
graph	openmp	2	64	8	8	40	32	0.93	1563804
graph	openmp	4	64	8	8	40	32	0.93	1457970
graph	pthread	2	64	8	8	40	32	0.94	1576530
graph	pthread	4	64	8	8	40	32	0.94	1582669
graph	openmp	2	64	8	16	40	16	0.92	1846005
graph	openmp	4	64	8	16	40	16	0.91	1581912
graph	pthread	2	64	8	16	40	16	0.92	1841721
graph	pthread	4	64	8	16	40	16	0.92	1662014
graph	openmp	2	64	8	16	40	32	0.96	1691279
graph	openmp	4	64	8	16	40	32	0.95	1450143

algorithm	thread_mode	threads	nlist	nprobe	hns_w_m	hns_w_efc	hns_w_ef	avg_recall	avg_lat_us
graph	pthread	2	64	8	16	40	32	0.96	1399732
graph	pthread	4	64	8	16	40	32	0.96	1684036
graph	openmp	2	128	16	8	40	16	0.85	38284
graph	openmp	4	128	16	8	40	16	0.83	103164
graph	pthread	2	128	16	8	40	16	0.88	257
graph	pthread	4	128	16	8	40	16	0.88	27751
graph	openmp	2	128	16	8	40	32	0.92	183
graph	openmp	4	128	16	8	40	32	0.91	22928
graph	pthread	2	128	16	8	40	32	0.95	242280
graph	pthread	4	128	16	8	40	32	0.95	302965
graph	openmp	2	128	16	8	100	16	0.87	174
graph	openmp	4	128	16	8	100	16	0.91	1631
graph	pthread	2	128	16	8	100	16	0.88	251
graph	pthread	4	128	16	8	100	16	0.88	77925
graph	openmp	2	128	16	8	100	32	0.97	12491
graph	openmp	4	128	16	8	100	32	0.97	32059
graph	pthread	2	128	16	8	100	32	0.98	250
graph	pthread	4	128	16	8	100	32	0.98	317
graph	openmp	2	128	16	16	40	16	0.88	12819
graph	openmp	4	128	16	16	40	16	0.90	5749
graph	pthread	2	128	16	16	40	16	0.90	263586
graph	pthread	4	128	16	16	40	16	0.90	1384
graph	openmp	2	128	16	16	40	32	0.95	103459
graph	openmp	4	128	16	16	40	32	0.96	806
graph	pthread	2	128	16	16	40	32	0.96	413226
graph	pthread	4	128	16	16	40	32	0.96	421852
graph	openmp	2	128	16	16	100	16	0.93	107674
graph	openmp	4	128	16	16	100	16	0.94	211
graph	pthread	2	128	16	16	100	16	0.92	221
graph	pthread	4	128	16	16	100	16	0.92	1022
graph	openmp	2	128	16	16	100	32	0.97	146265
graph	openmp	4	128	16	16	100	32	0.97	19896
graph	pthread	2	128	16	16	100	32	0.98	324
graph	pthread	4	128	16	16	100	32	0.98	1652
mpi_ivf	none	1	32	4	16	100	32	0.97	1828161
mpi_ivf	openmp	2	32	4	16	100	32	0.97	1658255
mpi_ivf	openmp	4	32	4	16	100	32	0.97	1267935
mpi_ivf	pthread	2	32	4	16	100	32	0.97	2534452
mpi_ivf	pthread	4	32	4	16	100	32	0.97	2552239
mpi_ivf	none	1	32	8	16	100	32	1.00	2031173
mpi_ivf	openmp	2	32	8	16	100	32	1.00	1344627
mpi_ivf	openmp	4	32	8	16	100	32	1.00	1040504
mpi_ivf	pthread	2	32	8	16	100	32	1.00	1912173
mpi_ivf	pthread	4	32	8	16	100	32	1.00	2266627
mpi_ivf	none	1	32	16	16	100	32	1.00	1907745
mpi_ivf	openmp	2	32	16	16	100	32	1.00	1553800
mpi_ivf	openmp	4	32	16	16	100	32	1.00	1212402
mpi_ivf	pthread	2	32	16	16	100	32	1.00	1975664
mpi_ivf	pthread	4	32	16	16	100	32	1.00	2078613
mpi_ivf	none	1	64	4	16	100	32	0.96	4015190
mpi_ivf	openmp	2	64	4	16	100	32	0.96	2315846
mpi_ivf	openmp	4	64	4	16	100	32	0.96	1889236
mpi_ivf	pthread	2	64	4	16	100	32	0.96	3936437
mpi_ivf	pthread	4	64	4	16	100	32	0.96	3805628
mpi_ivf	none	1	64	8	16	100	32	0.99	3530839
mpi_ivf	openmp	2	64	8	16	100	32	0.99	2206845
mpi_ivf	openmp	4	64	8	16	100	32	0.99	1856796
mpi_ivf	pthread	2	64	8	16	100	32	0.99	3322045
mpi_ivf	pthread	4	64	8	16	100	32	0.99	3041926
mpi_ivf	none	1	64	16	16	100	32	1.00	3126665
mpi_ivf	openmp	2	64	16	16	100	32	1.00	2211828
mpi_ivf	openmp	4	64	16	16	100	32	1.00	1787545
mpi_ivf	pthread	2	64	16	16	100	32	1.00	3187337
mpi_ivf	pthread	4	64	16	16	100	32	1.00	3704196
mpi_ivf	none	1	128	4	16	100	32	0.91	6965941
mpi_ivf	openmp	2	128	4	16	100	32	0.91	4107439

algorithm	thread_mode	threads	nlist	nprobe	hns_w_m	hns_w_efc	hns_w_ef	avg_recall	avg_lat_us
mpi_ivf	openmp	4	128	4	16	100	32	0.91	3048411
mpi_ivf	pthread	2	128	4	16	100	32	0.91	6105227
mpi_ivf	pthread	4	128	4	16	100	32	0.91	6705667
mpi_ivf	none	1	128	8	16	100	32	0.97	7322507
mpi_ivf	openmp	2	128	8	16	100	32	0.97	4644378
mpi_ivf	openmp	4	128	8	16	100	32	0.97	3521582
mpi_ivf	pthread	2	128	8	16	100	32	0.97	7500838
mpi_ivf	pthread	4	128	8	16	100	32	0.97	7773443
mpi_ivf	none	1	128	16	16	100	32	0.99	6817525
mpi_ivf	openmp	2	128	16	16	100	32	0.99	5399124
mpi_ivf	openmp	4	128	16	16	100	32	0.99	3685551
mpi_ivf	pthread	2	128	16	16	100	32	0.99	7276921
mpi_ivf	pthread	4	128	16	16	100	32	0.99	7943832

源代码仓库:

https://github.com/liuyuze-121/NKU-cpu-parallel-lab/tree/main/lab4_programming